

ECE 622: Modeling & Verification of Embedded Systems

Spring 2026: Lab 1: Reachability Analysis Report

Prof. Daniel Holcomb

Submitted by:	Shu-Lin Shih and Jaideep Khare
Student ID:	35199856 and 35193853
Lab No.:	1
Submission Date:	March 04, 2026

Overview

This lab explores explicit and symbolic reachability analysis for sequential systems. All experiments were performed on Ubuntu 16.04 using MiniSAT and PicoSAT.

Part A uses graph traversal and simulation-based reachability, while Part B implements SAT-based symbolic reachability through CNF encoding and transition relation unrolling. Part C extends the symbolic approach to counting reachable states.

1 Part A: Reachability by Graph Traversal

A.1: State Transition Graphs

For each design, we constructed the state transition graph by enumerating all states and evaluating the next-state for every possible input combination. The transitions were determined either directly from the gate-level schematic (`ex1`) and applying test vectors using simulation on the hdl file (`stoplight1`).

`ex1` Design

The design `ex1.v` contains two state bits (S_1, S_0) and one input A . Therefore the system has $2^2 = 4$ possible states and $2 \times 4 = 8$ possible (A, S_1, S_0) combinations.

Using the gate-level schematic of `ex1.v`, we evaluated the next-state logic (NS_1, NS_0) for each combination. The resulting transition truth table is shown in Table 1.

A	S1	S0	NS1	NS0
0	0	0	0	1
1	0	0	0	1
0	0	1	0	1
1	0	1	0	1
0	1	0	0	1
1	1	0	0	1
0	1	1	0	0
1	1	1	1	0

Table 1: State transition truth table for `ex1.v`

From Table 1, rows that lead to identical state transitions were grouped together and represented as labeled edges in the state transition graph. The resulting FSM is shown in Fig. 1. The initial state is 00.

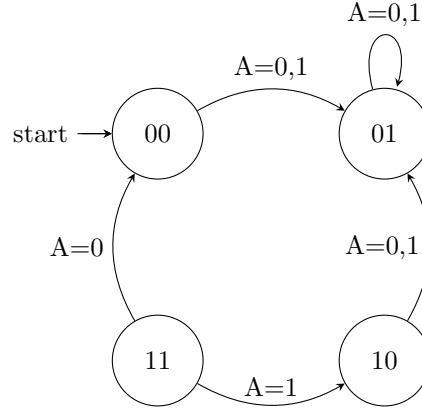


Figure 1: State transition graph for `ex1.v`, only $\{0,1\}$ is reachable from initial

stoplight1 Design

The design `stoplight1.v` contains four state bits, resulting in $2^4 = 16$ possible states. For each state $S \in \{0000, 0001, \dots, 1111\}$ we evaluated the next state for both values of the pedestrian input signal ($\text{Ped}=0$ and $\text{Ped}=1$) by simulating the design in AMD Vivado HDL Simulator screenshot as shown.

Using the transition relations obtained from simulation results (Fig. 2), we constructed the FSM shown in Fig. 3. Each node represents a 4-bit state and each directed edge is labeled with the corresponding input value of Ped .

S=0000	Ped=0 -> 0001	Ped=1 -> 0001
S=0001	Ped=0 -> 0010	Ped=1 -> 0010
S=0010	Ped=0 -> 0011	Ped=1 -> 0011
S=0011	Ped=0 -> 0100	Ped=1 -> 0110
S=0100	Ped=0 -> 0101	Ped=1 -> 0111
S=0101	Ped=0 -> 0101	Ped=1 -> 1000
S=0110	Ped=0 -> 0111	Ped=1 -> 0111
S=0111	Ped=0 -> 1000	Ped=1 -> 1000
S=1000	Ped=0 -> 1001	Ped=1 -> 1001
S=1001	Ped=0 -> 0000	Ped=1 -> 0000
S=1010	Ped=0 -> 1011	Ped=1 -> 1011
S=1011	Ped=0 -> 0000	Ped=1 -> 0010
S=1100	Ped=0 -> 1001	Ped=1 -> 1011
S=1101	Ped=0 -> 0000	Ped=1 -> 0000
S=1110	Ped=0 -> 1011	Ped=1 -> 1011
S=1111	Ped=0 -> 0000	Ped=1 -> 0000

Figure 2: Screenshot for `stoplight1.v` testbench simulation

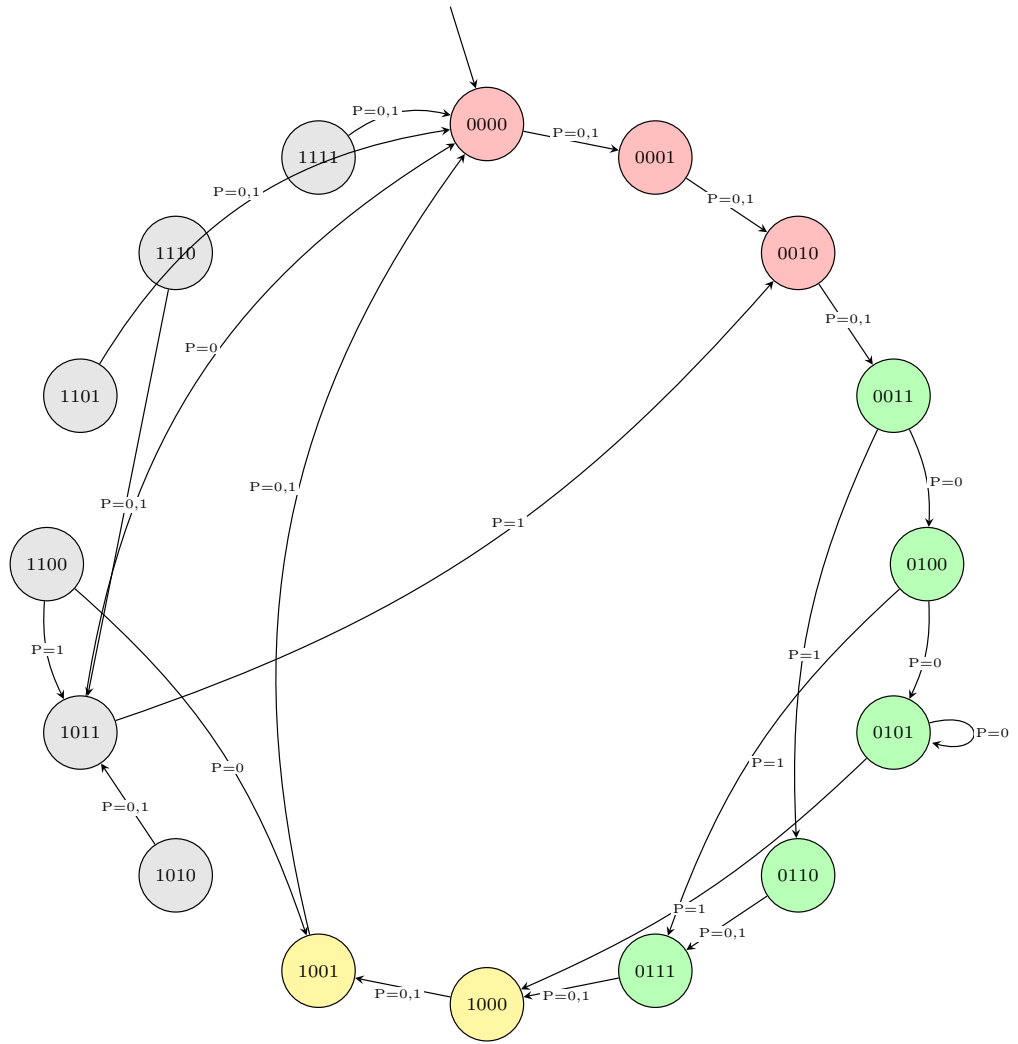


Figure 3: Complete state transition graph for **stoplight1.v**. States are color-coded for readability. Unreachability is indicated using grey-colored state circles with respect to given initial state 0000.

A.2: Target Reachability and Shortest Path

ex1: target state 11 is unreachable. From the state transition graph of **ex1.v**, the initial state is 00 and it transitions to 01 for both $A = 0$ and $A = 1$. State 01 then self-loops for both $A = 0$ and $A = 1$, so the machine becomes trapped in 01 after at most one transition. Therefore, state 11 is not reachable from the initial state 00 under any input sequence.

stoplight1: shortest path to target state 0100. From the state transition graph of **stoplight1.v**, the following shortest path (4 transitions) reaches the target state 0100 from the initial state 0000:

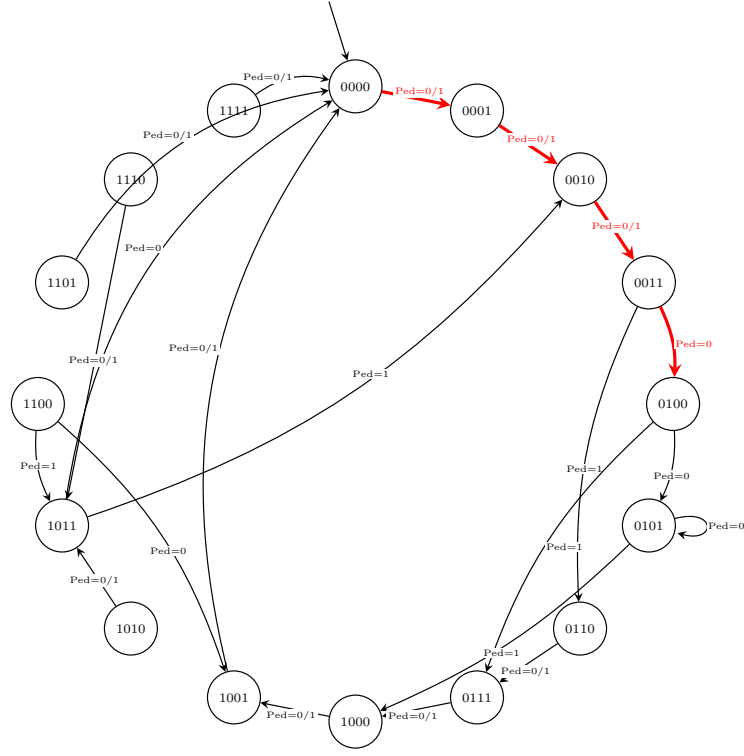


Figure 4: State transition graph for `stoplight1.v`. The shortest path from 0000 to target 0100 is highlighted in red.

A.3: States Reachable Within 20 transitions

For `ex1.v`, the reachable set saturates immediately: from initial 00, the machine reaches 01 in one step and then self-loops forever, so the number of reachable states within i transitions is constantly 2 for $i \geq 1$.

$i \in \{1, 2, \dots, 20\}$	# Reachable states within i transitions (<code>ex1</code>)
1	1 (not counting initial state)
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	1
13	1
14	1
15	1
16	1
17	1
18	1
19	1
20	1

Table 2: Cumulative reachable-state counts for `ex1.v`, from the initial state $\{00\}$.

For `stoplight1.v`, the number of reachable states grows over the first few transitions and then saturates at 10 states. The increase in reachable states from 3 to 5 (when j increases from 3 to 4) occurs because different values of `Ped` introduce branching transitions that can shorten the green-light phase. This branching allows the

system to reach additional states that are not accessible under a single deterministic input sequence. Overall, the design contains a total of ten valid states: three Red states, five Green states, and two Yellow states. All of these states become reachable once the exploration depth satisfies $i \geq 8$, after which the reachable set saturates and no new states are discovered.

$i \in \{1, 2, \dots, 20\}$	# Reachable states within i transitions (stoplight1)
1	1
2	2
3	3
4	5
5	7
6	8
7	9
8	10
9	10
10	10
11	10
12	10
13	10
14	10
15	10
16	10
17	10
18	10
19	10
20	10

Table 3: Cumulative reachable-state counts for `stoplight1.v`, from the initial state `{0000}`.

A.4: Random Execution Using Verilog Testbench

A Verilog testbench was written to explore reachability using random simulation. The testbench (code snapshot included in appendix) performs the following steps:

- Forces the initial state of the FSM to the reset state.
- Applies random input values every clock cycle.
- Checks at each cycle whether the target state has been reached.
- Terminates immediately if the target state is reached.
- Stops the experiment after 100,000 cycles if the target state is never observed.

Observed simulation outcomes:

- **ex1:** Target state 11 was **not reached** within 100,000 cycles (timeout).
- **ex2:** Target state 111111111111 was **not reached** within 100,000 cycles (timeout).
- **stoplight1:** Target state 0100 was **reached at cycle 52**.
- **stoplight2:** Target state 01000 was **reached at cycle 1357**.

The simulation logs confirming these results are shown in the figures below.

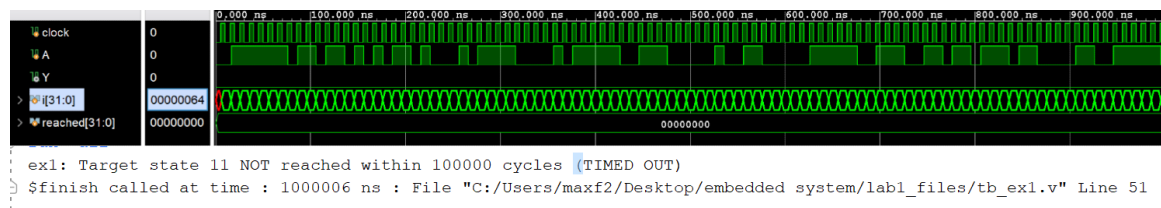


Figure 5: Waveform for **ex1**. Target state 11 was not reached within the simulation limit of 100,000 cycles.

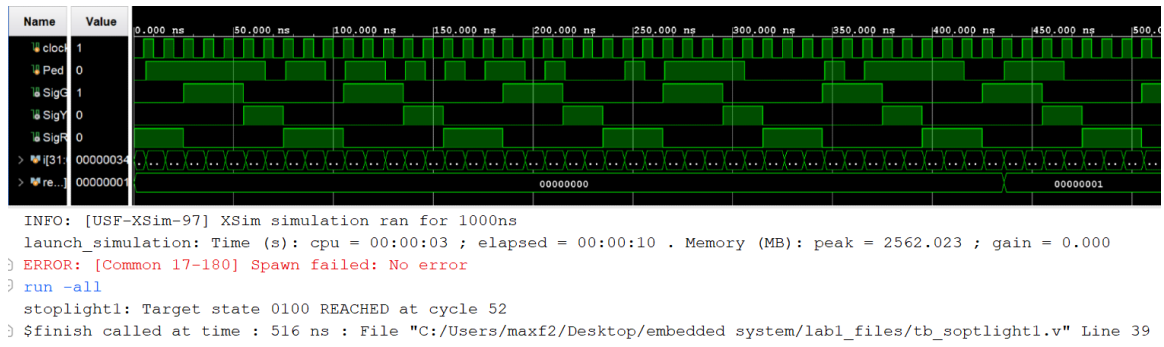


Figure 6: Waveform for `stoplight1`. Target state 0100 was reached during random simulation.

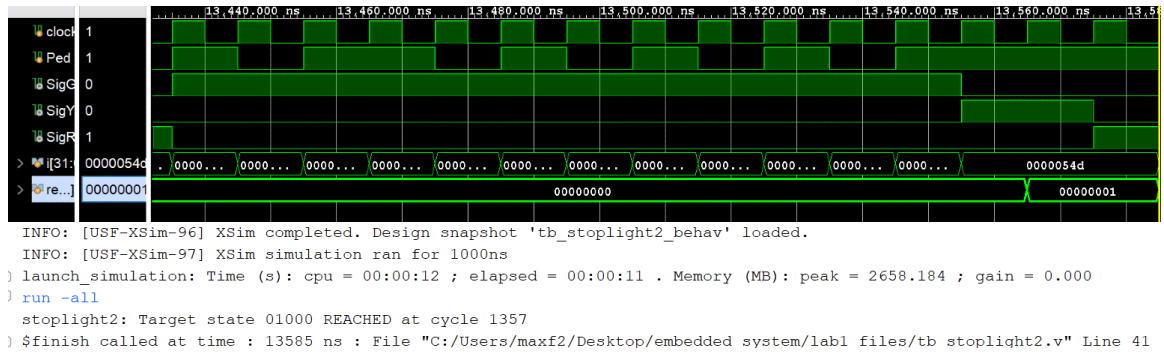


Figure 7: Waveform for `stoplight2`. Target state 01000 was reached at cycle 1357.



Figure 8: Waveform for `ex2`. Target state was not reached during the simulation interval.

We conclude that while this approach provides a practical way to observe reachability behavior in simulation, it does not guarantee exploration of all possible input sequences.

A.5: Consistency Between Graph Analysis and Simulation

For `stoplight1`, the simulation result is found to be consistent with the reachability predicted by the transition graph.

Analysis on different approaches (A.2 and A.4):

From the graph analysis in A.2, the target state 0100 is reachable from the initial state, and the shortest path requires at least four transitions. In the random simulation experiment of A.4, the target state was reached at cycle 52. The larger number of cycles occurs because the simulation uses randomly generated inputs.

The FSM may take alternative branches in the transition graph (for example transitions through state 0110) before eventually following the sequence that leads to the target state.

A finding that would indicate an error in the earlier analysis would be if the simulation *never* reached state 0100 even after a very large number of cycles. Since the transition graph shows that this state is reachable, such a result would imply that either the graph analysis in A.2 or the simulation setup in A.4 was incorrect.

Discussion: These results illustrate an important limitation of random simulation. While random execution over many cycles can provide evidence about reachability, it cannot formally guarantee that a state is unreachable. For complex circuits, simulation may require extremely long runtimes before a rare input sequence appears.

Therefore, more systematic techniques such as symbolic execution, SAT-based bounded model checking, or formal reachability analysis are required to accurately determine circuit behavior and prove whether a state is reachable or unreachable.

2 Part B: SAT-Based Symbolic Reachability

In this part, we implemented symbolic reachability using CNF encoding and SAT solving.

Program Overview

Our C++ program performs the following steps:

1. Parse Verilog netlist.
2. Generate Boolean variables for each node.
3. Encode AND and NOT gates into CNF.
4. Unroll transition relation k times.
5. Add initial state constraints.
6. Add target state constraint at transition k .
7. Output DIMACS CNF and node mapping.

B.1: ex1 Unrolled 2 Times

Using the SAT-based reachability tool, the FSM in `ex1.v` was unrolled for $k = 2$ transitions starting from the initial state 00. Each possible target state was checked independently.

Target state 11

The SAT solver returned **UNSAT**, meaning no assignment of inputs allows the FSM to reach state 11 within two transitions.

This is consistent with the transition graph derived in Section A.1, where the FSM moves from 00 to 01 and remains trapped in the self-loop at 01.

```

jdnotlurking@14art-he:~/622modeling/lab1$ ./lab1_parser ./verilog_src/ex1.v 11 2 00 results_ex1 minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:           18
| Number of clauses:            30
| Parse time:                   0.00 s
| Simplification time:          0.00 s
|
|=====
Solved by simplification
restarts      : 0
conflicts    : 0                (0 /sec)
decisions    : 0                (-nan % random) (0 /sec)
propagations : 10               (5609 /sec)
conflict literals : 0          (-nan % deleted)
Memory used  : 22.00 MB
CPU time     : 0.001783 s

UNSATISFIABLE
WARNING: minisat returned non-zero exit code 5120
Wrote results_ex1/out.dimacs and results_ex1/out.nodes
State regs inferred (target bit order assumed to match this order): S1 S0
jdnotlurking@14art-he:~/622modeling/lab1$

```

Figure 9: SAT solver output for target state 11 (UNSAT).

Target state 00

The solver returned **UNSAT**. Once the FSM leaves state 00, it cannot return to it within the given transition structure.

```

jdnotlurking@14art-he:~/622modeling/lab1$ ./lab1_parser ./verilog_src/ex1.v 00 2 00 results_ex1 minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:           18
| Number of clauses:            30
| Parse time:                   0.00 s
| Simplification time:          0.00 s
|
|=====
Solved by simplification
restarts      : 0
conflicts    : 0                (0 /sec)
decisions    : 0                (-nan % random) (0 /sec)
propagations : 10               (5405 /sec)
conflict literals : 0          (-nan % deleted)
Memory used  : 22.00 MB
CPU time     : 0.00185 s

UNSATISFIABLE
WARNING: minisat returned non-zero exit code 5120
Wrote results_ex1/out.dimacs and results_ex1/out.nodes
State regs inferred (target bit order assumed to match this order): S1 S0
jdnotlurking@14art-he:~/622modeling/lab1$

```

Figure 10: SAT solver output for target state 00 (UNSAT).

Target state 01

The solver returned **SAT**. This state is reachable from the initial state in one transition and contains a self-loop, meaning it remains reachable for any unrolling depth $k \geq 1$.


```

jdnotlurking@14art-he:~/622modeling/lab1$ ./lab1_parser ./verilog_src/ex1.v 01 2 00 results_ex1 minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:      18
| Number of clauses:       30
| Parse time:              0.00 s
| Eliminated clauses:      0.00 Mb
| Simplification time:     0.00 s
|
|===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress |
| | Vars | Clauses | Literals | Limit | Clauses | Lit/Cl |
|=====
restarts      : 1
conflicts     : 0 (0 /sec)
decisions     : 1 (0.00 % random) (717 /sec)
propagations  : 14 (10036 /sec)
conflict literals : 0 (-nan % deleted)
Memory used   : 22.00 MB
CPU time      : 0.001395 s

SATISFIABLE
WARNING: minisat returned non-zero exit code 2560
Wrote results_ex1/out.dimacs and results_ex1/out.nodes
State regs inferred (target bit order assumed to match this order): S1 S0
jdnotlurking@14art-he:~/622modeling/lab1$

```

Figure 11: SAT solver output showing reachability of state 01.

Target state 10

The solver returned **UNSAT**, confirming that state 10 is not reachable from the initial state 00 within two transitions.

```

jdnotlurking@14art-he:~/622modeling/lab1$ ./lab1_parser ./verilog_src/ex1.v 10 2 00 results_ex1 minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:      18
| Number of clauses:       30
| Parse time:              0.00 s
| Simplification time:     0.00 s
|
|=====
Solved by simplification
restarts      : 0
conflicts     : 0 (0 /sec)
decisions     : 0 (-nan % random) (0 /sec)
propagations  : 10 (4850 /sec)
conflict literals : 0 (-nan % deleted)
Memory used   : 22.00 MB
CPU time      : 0.002062 s

UNSATISFIABLE
WARNING: minisat returned non-zero exit code 5120
Wrote results_ex1/out.dimacs and results_ex1/out.nodes
State regs inferred (target bit order assumed to match this order): S1 S0
jdnotlurking@14art-he:~/622modeling/lab1$

```

Figure 12: SAT solver output for target state 10 (UNSAT).

B.2: 15-Step Reachability Analysis

The transition relation of each design was unrolled for $k = 15$ steps and encoded into a CNF formula. The resulting DIMACS file was solved using MiniSAT to determine whether the specified target state is reachable from the initial state.

Design	Reachable within 15 steps?	SAT Result
ex1	No	UNSAT
ex2	No	UNSAT
ex3	Yes	SAT
ex4	No	UNSAT
ex5	No	UNSAT
stoplight1	Yes	SAT
stoplight2	Yes	SAT

Table 4: Reachability results for 15-step unrolling (same for minisat and picosat)

Design	#Variables	#Clauses	MiniSAT CPU Time (s)	PicoSAT Time (s)
ex1	122	225	0.001926	0.0
ex2	3540	8385	0.006233	0.0
ex3	9074	22785	0.011038	0.0
ex4	7577	18210	0.003554	0.0
ex5	3444	8205	0.004606	0.0
stoplight1	934	2310	0.003060	0.0
stoplight2	1715	4455	0.004533	0.0

Table 5: SAT solver statistics for 15-step reachability (MiniSAT vs PicoSAT)

A screenshot showing run time measurements from picosat is attached in Appendix (Fig 21). **ex1** Target state 11 is not reachable within 15 transitions.

```
jdnotlurking@i4art-he:~/622modeling/lab1_2$ ./lab1_parser verilog_src/ex1.v 11 15 00 minisat /usr/bin/minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
| Number of variables:      122
| Number of clauses:       225
| Parse time:              0.00 s
| Simplification time:     0.00 s
=====
Solved by simplification
restarts      : 0
conflicts    : 0              (0 /sec)
decisions    : 0              (-nan % random) (0 /sec)
propagations : 10             (5192 /sec)
conflict literals : 0          (-nan % deleted)
Memory used  : 22.00 MB
CPU time     : 0.001926 s

UNSATISFIABLE
WARNING: minisat returned non-zero exit code 5120
Wrote minisat/out.dimacs and minisat/out.nodes
State regs inferred (target bit order assumed to match this order): S1 S0
```

Figure 13: SAT solver result for **ex1** (UNSAT).

ex2

Target state consisting of all ones was not reachable within the 15-step bound.

```
jdnotlurking@i4art-he:~/622modeling/lab1_2$ ./lab1_parser verilog_src/ex2.v 1111111111111111 15 00000000000000 minisat /usr/bin/minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
| Number of variables:      3540
| Number of clauses:       8385
| Parse time:              0.00 s
| Eliminated clauses:      0.00 Mb
| Simplification time:     0.01 s
=====
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNED | Progress | | |
|---|---|---|---|---|---|
| Vars | Clauses | Literals | Limit | Clauses | Lit/Cl |
|-----|-----|-----|-----|-----|
restarts      : 1
conflicts    : 2              (321 /sec)
decisions    : 3              (0.00 % random) (481 /sec)
propagations : 584            (93695 /sec)
conflict literals : 1          (0.00 % deleted)
Memory used  : 23.00 MB
CPU time     : 0.006233 s

UNSATISFIABLE
WARNING: minisat returned non-zero exit code 5120
Wrote minisat/out.dimacs and minisat/out.nodes
State regs inferred (target bit order assumed to match this order): S14 S13 S12 S11 S10 S9 S8 S7 S6 S5 S4 S3 S2 S1 S0
```

Figure 14: SAT solver result for **ex2** (UNSAT).

ex3

The solver returned **SAT**, indicating that the target state is reachable within the unrolling bound.

```

jdnotturking@14art-he:~/622modeling/lab1_25$ ./lab1_parser verillog_src/ex3.v 0000000001011010000011000001 15 00000000000000000000000000000000 minisat /usr/bin/minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:      9074
| Number of clauses:       22785
| Parse time:              0.00 s
| Eliminated clauses:      0.19 Mb
| Simplification time:     0.00 s
|
|===== [ Search Statistics ]=====
| Conflicts | ORIGINAL | LEARNED | Progress |
|-----|-----|-----|-----|
| restarts  | : 1
| conflicts  | : 37      (3352 /sec)
| decisions | : 263     (0.00 % random) (23827 /sec)
| propagations | : 3617   (327686 /sec)
| conflict literals | : 145    (0.45 % deleted)
| Memory used | : 25.00 MB
| CPU time   | : 0.011038 s
|
| SATISFIABLE
| WARNING: minisat returned non-zero exit code 2560
| Wrote minisat/out.dimacs and minisat/out.nodes
| State regs inferred (target bit order assumed to match this order): S28 S27 S26 S25 S24 S23 S22 S21 S20 S19 S18 S17 S16 S15 S14 S13 S12 S11 S10 S9 S8 S7 S6 S5 S4 S3 S2 S1 S0
jdnotturking@14art-he:~/622modeling/lab1_25$

```

Figure 15: SAT solver result for **ex3** (SAT).

ex4

The solver returned **UNSAT**, indicating the target state is not reachable within 15 transitions.

```

jdnotturking@14art-he:~/622modeling/lab1_25$ ./lab1_parser verillog_src/ex4.v 00000000000000000000000000000000 15 00000000000000000000000000000000 minisat /usr/bin/minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:      7577
| Number of clauses:       18210
| Parse time:              0.00 s
| Eliminated clauses:      0.05 Mb
| Simplification time:     0.00 s
|
| Solved by simplification
| restarts : 0
| conflicts : 0 (0 /sec)
| decisions : 0 (-nan % random) (0 /sec)
| propagations : 4956 (1394485 /sec)
| conflict literals : 0 (-nan % deleted)
| Memory used : 24.00 MB
| CPU time : 0.003554 s
|
| UNSATISFIABLE
| WARNING: minisat returned non-zero exit code 5120
| Wrote minisat/out.dimacs and minisat/out.nodes
| State regs inferred (target bit order assumed to match this order): S31 S30 S29 S28 S27 S26 S25 S24 S23 S22 S21 S20 S19 S18 S17 S16 S15 S14 S13 S12 S11 S10 S9 S8 S7 S6 S5 S4 S3 S2 S1 S0
jdnotturking@14art-he:~/622modeling/lab1_25$

```

Figure 16: SAT solver result for **ex4** (UNSAT).

ex5

The solver returned **UNSAT**, meaning the specified target state cannot be reached within the bound.

```

jdnotturking@14art-he:~/622modeling/lab1_25$ ./lab1_parser verillog_src/ex5.v 000110111 15 000000000 minisat /usr/bin/minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:      3444
| Number of clauses:       8205
| Parse time:              0.00 s
| Simplification time:     0.00 s
|
| Solved by simplification
| restarts : 0
| conflicts : 0 (0 /sec)
| decisions : 0 (-nan % random) (0 /sec)
| propagations : 245 (53191 /sec)
| conflict literals : 0 (-nan % deleted)
| Memory used : 23.00 MB
| CPU time : 0.004606 s
|
| UNSATISFIABLE
| WARNING: minisat returned non-zero exit code 5120
| Wrote minisat/out.dimacs and minisat/out.nodes
| State regs inferred (target bit order assumed to match this order): S8 S7 S6 S5 S4 S3 S2 S1 S0
jdnotturking@14art-he:~/622modeling/lab1_25$

```

Figure 17: SAT solver result for **ex5** (UNSAT).

stoplight1

The solver returned **SAT**, confirming that the target state 0100 is reachable within the 15-step bound.

```

jdnotlurking@i4art-he:~/622modeling/lab1_2$ ./lab1_parser verilog_src/stoplight1.v 0100 15 0000 minisat /usr/bin/minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:      934
| Number of clauses:       2310
| Parse time:              0.00 s
| Eliminated clauses:      0.02 Mb
| Simplification time:     0.00 s
|
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress | | |
|---|---|---|---|---|---|
| Vars | Clauses | Literals | Limit | Clauses | Lit/Cl |
|-----|-----|-----|-----|-----|
restarts      : 1
conflicts     : 11          (3595 /sec)
decisions     : 32          (0.00 % random) (10458 /sec)
propagations  : 731        (238889 /sec)
conflict literals : 24      (4.00 % deleted)
Memory used   : 22.00 MB
CPU time      : 0.00306 s

SATISFIABLE
WARNING: minisat returned non-zero exit code 2560
Wrote minisat/out.dimacs and minisat/out.nodes
State regs inferred (target bit order assumed to match this order): S3 S2 S1 S0
jdnotlurking@i4art-he:~/622modeling/lab1_2$

```

Figure 18: SAT solver result for `stoplight1` (SAT).

stoplight2

The solver returned **SAT**, confirming that the target state 01000 is reachable.

```

jdnotlurking@i4art-he:~/622modeling/lab1_2$ ./lab1_parser verilog_src/stoplight2.v 01000 15 00000 minisat /usr/bin/minisat
WARNING: for repeatability, setting FPU to use double precision
===== [ Problem Statistics ] =====
|
| Number of variables:      1715
| Number of clauses:       4455
| Parse time:              0.00 s
| Eliminated clauses:      0.03 Mb
| Simplification time:     0.00 s
|
===== [ Search Statistics ] =====
| Conflicts | ORIGINAL | LEARNT | Progress | | |
|---|---|---|---|---|---|
| Vars | Clauses | Literals | Limit | Clauses | Lit/Cl |
|-----|-----|-----|-----|-----|
restarts      : 1
conflicts     : 46          (10148 /sec)
decisions     : 70          (0.00 % random) (15442 /sec)
propagations  : 2694        (594308 /sec)
conflict literals : 252      (23.87 % deleted)
Memory used   : 22.00 MB
CPU time      : 0.004533 s

SATISFIABLE
WARNING: minisat returned non-zero exit code 2560
Wrote minisat/out.dimacs and minisat/out.nodes
State regs inferred (target bit order assumed to match this order): S4 S3 S2 S1 S0
jdnotlurking@i4art-he:~/622modeling/lab1_2$

```

Figure 19: SAT solver result for `stoplight2` (SAT).

B.3: stoplight1 Reachable on i -th Transition (1–32)

To determine for which transition index i the target state becomes reachable, we invoked the SAT-based reachability tool repeatedly while varying the unrolling depth from $i = 1$ to $i = 32$. For each value of i , the transition relation was unrolled i times and the solver was asked whether the target state 0100 could be reached from the initial state 0000.

The SAT solver reported **UNSAT** for values $i < 12, i \neq 4$ and has **SAT** solutions for values $i \geq 12$ and $i = 4$.

Reachable indices:

$$i = \{4, 12, 13, \dots, 31, 32\}$$

This means that although the shortest path in the state graph may require fewer transitions, the bounded model checking formulation detects reachability starting at $i = 12$.

B.4: stoplight2 Comparison with Random Simulation

Using the same symbolic reachability procedure, the solver was invoked for increasing transition depths i from 1 to 32 for the circuit `stoplight2.v`. The target state considered was 01000.

The solver returned **UNSAT** for all values $i < 14$ and **SAT** for all values $i \geq 14$.

Reachable indices:

$$i = \{14, 15, 16, \dots, 31, 32\}$$

Comparison with A.4 (Random Simulation)

In Section A.4, random simulation reached the same target state after 1357 clock cycles. The symbolic SAT-based analysis here determines the earliest transition depth at which the state becomes reachable, independent of any particular input sequence.

Thus, symbolic reachability analysis provides a deterministic bound on reachability, while random simulation may take significantly longer to encounter the required sequence of inputs.

B.5: States Reachable on the 17th Transition (stoplight2)

To determine the set of states reachable exactly at transition $k = 17$, we computed the set $\delta^{17}(S_0)$ using SAT-based reachability analysis.

Approach

- The transition relation was unrolled for $k = 17$ steps starting from the fixed initial state.
- For each candidate state s , unit clauses were added constraining the state variables at time step 17 to equal s .
- The SAT solver was invoked to determine whether a satisfying assignment exists.
- If the solver returned SAT, this indicates that there exists an input sequence of length 17 that drives the FSM from the initial state to s .
- This procedure was repeated for all $2^5 = 32$ possible states of the FSM.

Using this method, the following **19 states** were found to be reachable at the 17th transition:

$$\{00000, 00001, 00010, 00011, 00100, 00101, 00110, 00111, 01000, 01001, \\ 10000, 10001, 10010, 10011, 10100, 10101, 10110, 10111, 11000\}$$

These states represent all configurations for which a valid sequence of 17 inputs exists that leads from the initial state to the specified state.

3 Part C: Counting Reachable States (ECE 622)

C.1: Reachable States in ex5 and ex1

Table 6: Part C (ex5): Number of reachable states at transition i

i	SAT count
1	8/512
2	14/512
3	18/512
4	22/512
5	26/512
6	28/512
7	28/512
8	28/512
9	28/512
10	28/512
11	28/512
12	28/512
13	28/512
14	28/512
15	28/512
16	28/512
17	28/512
18	28/512
19	28/512
20	28/512

Table 7: Part C (ex1): Reachable target states at transition i (MiniSat SAT results)

i	SAT count
1	1/4
2	1/4
3	1/4
4	1/4
5	1/4
6	1/4
7	1/4
8	1/4
9	1/4
10	1/4
11	1/4
12	1/4
13	1/4
14	1/4
15	1/4
16	1/4
17	1/4
18	1/4
19	1/4
20	1/4

C.2: Approach Description

Our approach to counting reachable states:

- Iteratively unrolled transition relation.

- Used SAT solver to check existence.
- Added blocking clauses to enumerate unique states.
- Maintained cumulative set of reachable states.

Code submitted separately on canvas. (`minisat_reach_table.cpp` and `partc_hashmap.sh`)

Use of Tools

We used the following tools in completing this lab:

- MiniSAT / PicoSAT on Ubuntu 14.
- AMD Vivado 2025.1 on Windows 10.
- C++ for CNF generation
- TikZ for graph visualization
- ChatGPT for assistance in structuring Lab report.

AI prompts and descriptions are included in the appendix.

Appendix

A. Testbench Used for Random Simulation (A.4)

```
1 `timescale 1ns/1ps
2
3 module tb_stoplight1;
4     reg clock, Ped;
5     wire SigG, SigY, SigR;
6     integer i;
7     integer reached;
8
9     stoplight1 spotlight1 (
10         .clock(clock),
11         .Ped(Ped),
12         .SigG(SigG),
13         .SigY(SigY),
14         .SigR(SigR)
15     );
16
17     initial clock = 0;
18     always #5 clock = ~clock;
19
20     initial begin
21         reached = 0;
22         Ped = 0;
23         spotlight1.S3 = 0;
24         spotlight1.S2 = 0;
25         spotlight1.S1 = 0;
26         spotlight1.S0 = 0;
27
28         for (i = 1; i <= 100000; i = i + 1) begin
29             Ped = $random % 2;
30             @(posedge clock);
31
32             // Target:0100
33             if (spotlight1.S3 == 0 && spotlight1.S2 == 1 && spotlight1.S1 == 0 && spotlight1.S0 == 0) begin
34                 $display("stoplight1: Target state 0100 REACHED at cycle %0d", i);
35                 reached = 1;
36                 @(posedge clock);
37                 @(posedge clock);
38                 $finish;
39             end
40         end
41
42         if (reached == 0)
43             $display("stoplight1: Target state 0100 NOT reached within 100000 cycles (TIMED OUT)");
44
45         $finish;
46     end
47 endmodule
```

Figure 20: Verilog testbench used for random execution experiments in Section A.4.

The testbench initializes the FSM to its starting state and applies randomized values to the input signal Ped at each clock cycle. The simulation terminates when the target state is reached or after a fixed number of cycles.

```
jdnotlurking@14art-he:~/622modeling/lab1_2$ picosat b2ex1_mini/out.dimacs -v > b2ex1_mini/out.sat
jdnotlurking@14art-he:~/622modeling/lab1_2$ picosat b2ex2_mini/out.dimacs -v > b2ex2_mini/out.sat
jdnotlurking@14art-he:~/622modeling/lab1_2$ picosat b2ex3_mini/out.dimacs -v > b2ex3_mini/out.sat
jdnotlurking@14art-he:~/622modeling/lab1_2$ picosat b2ex4_mini/out.dimacs -v > b2ex4_mini/out.sat
jdnotlurking@14art-he:~/622modeling/lab1_2$ picosat b2ex5_mini/out.dimacs -v > b2ex5_mini/out.sat
jdnotlurking@14art-he:~/622modeling/lab1_2$ picosat b2stoplight1_mini/out.dimacs -v > b2stoplight1_mini/out.sat
jdnotlurking@14art-he:~/622modeling/lab1_2$ picosat b2stoplight2_mini/out.dimacs -v > b2stoplight2_mini/out.sat
jdnotlurking@14art-he:~/622modeling/lab1_2$ grep *run time* b2*/out.sat
grep: time*: No such file or directory
jdnotlurking@14art-he:~/622modeling/lab1_2$ grep "run time" b2*/out.sat
b2ex1_mini/out.sat:c 0.0 seconds total run time
b2ex2_mini/out.sat:c 0.0 seconds total run time
b2ex3_mini/out.sat:c 0.0 seconds total run time
b2ex4_mini/out.sat:c 0.0 seconds total run time
b2ex5_mini/out.sat:c 0.0 seconds total run time
b2stoplight1_mini/out.sat:c 0.0 seconds total run time
b2stoplight2_mini/out.sat:c 0.0 seconds total run time
jdnotlurking@14art-he:~/622modeling/lab1_2$
```

Figure 21: Picosat solver run time measurements

B. Prompts Used for Tool Assistance

As required by the lab instructions, we document the prompts used when employing AI-based tools to assist with diagram generation, formatting, and documentation. AI assistance was used only as a productivity aid for generating templates or converting information into LaTeX format; all logic analysis, reachability reasoning, and correctness verification were performed manually.

B.1 Prompt for Generating the ex1 FSM Graph

Prompt Used:

Generate a TikZ template for a 4-state FSM graph with labeled transitions that I can adapt for reachability analysis in Verilog.

Purpose: This prompt was used to generate an initial TikZ template for a small FSM diagram. The final diagram in the report therefore reflects the manually derived state transition relation.

B.2 Prompt for Generating the stoplight1 FSM Layout

Prompt Used:

Generate a TikZ diagram for a 16-state FSM arranged in a circular layout. Transitions should be labeled with Ped input values, with white background labels to keep them readable. Allow color-coded states according to a reference table and highlight a specific path of transitions in red.

Purpose:

This prompt was used to obtain a structural template for visualizing a larger FSM containing 16 possible states.

B.3 Prompt for Converting Screenshots into LaTeX Tables

Prompt Used:

Convert the numerical data shown in the attached screenshot into a properly formatted LaTeX table suitable for inclusion in an academic report.

Purpose: ChatGPT is very quick at processing screenshot (of logs / other documents) and converting them to report-friendly text. Hence we used prompts to convert tabulated experimental results from screenshots into LaTeX table format. The resulting tables were then reviewed and manually verified to ensure consistency with the reachability experiments. This assisted in producing Tables 2 and 3 in Section A.3.

C. Additional Notes

- GitHub repository containing all source code and scripts used for this lab:
https://github.com/JDKhare/UMass_ECE622_Labs/tree/main/lab1